

Sumário

1	Mecanismos de abstração	2
1.1	Classes abstratas	2
1.1.1	Métodos abstratos	3
1.2	Interfaces	4
1.3	Métodos estáticos e default	5
1.3.1	Herança múltipla	5
1.4	Exemplo	5
1.5	Comparação entre classes abstratas e interfaces	9
	Referências Bibliográficas	14

1 Mecanismos de abstração

Na programação Orientada a Objetos utilizamos o termo **concreto(a)** para definir classes e métodos que são implementados de forma completa.

Todas as classes e métodos criados até então eram do tipo concretos, ou seja, tanto as super classes como as subclasses implementavam todos os métodos. Além disso, as classes concretas participavam do sistema diretamente.

Observação

Em Java, costumamos chamar as classes concretas de POJO (*Plain Old Java Object*). Teoricamente é possível ter POJO não concreto, mas na prática isso não é comum. Características de um POJO:

- Sem herança forçada: Não estende classes de frameworks (como `HttpServlet` ou `ActionForm`).
- Sem interfaces obrigatórias: Não implementa interfaces complexas de terceiros.
- Sem anotações obrigatórias: Não exige anotações predefinidas para funcionar.
- Simplicidade: Contém apenas atributos, métodos de acesso (getters/setters) e regras de negócio simples.

Entretanto, no desenvolvimento de software, algumas super classes não fazem sentido serem instanciadas, no exemplo da clinica veterinária, não vamos criar um objeto **pessoa** e sim veterinário ou cliente.

Esse tipo de classe pode servir, em alguns casos, apenas de modelos para as subclasses ou fazer parte do **encapsulamento**.

1.1 Classes abstratas

Uma classe abstrata serve como uma base estrutural que não deve ser utilizada de forma isolada. Por representar conceitos muito genéricos, ela permanece incompleta por design. Assim, o papel da classe abstrata é definir um “molde” que precisa ser concretizado pelas subclasses, as quais herdaram e preencherão as lacunas de implementação necessárias para que o objeto possa ser funcional (Leite et al. (2016)).

Essas classe elas têm uma característica importante: **não podem ser instanciadas**. Ou seja, não podemos criar objetos diretamente a partir delas (Leite et al. (2016)). Logo,

essas classes devem estar no topo da hierarquia de classes.

Observação

Classes abstratas não podem ser instanciadas. Logo elas devem estar no topo da hierarquia de classes.

Para criarmos uma classe abstrata em Java utilizamos a palavra **abstract** antes do **class** e do nome da classe.

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composicao: Pessoa "tem um" Endereco
    private Endereco endereco;

    public Pessoa(String nome, String cpf, String email, String
        rua, String cidade) {
        this.nome = nome;
        this.cpf=cpf;
        this.email=email;
        this.endereco = new Endereco(rua, cidade);
    }
}
```

1.1.1 Métodos abstratos

Podemos também definir métodos como abstratos. A ideia é para que método também sirva de molde. Para isso, **ele não deve possuir uma implementação**, mas sim apenas a definição de sua assinatura (Leite et al. (2016)). Portanto as subclasses devem implementar esses métodos, utilizando a sobreescrita **@Override**.

Observação

- Uma classe abstrata não precisa necessariamente ter um método abstrato.
- Um método abstrato deve ser implementado nas classes filhas utilizando a notação **@Override**.

Veja o exemplo de uso de um método abstrato.

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composicao: Pessoa "tem um" Endereco
```

```

private Endereco endereco;

public Pessoa(String nome, String cpf, String email, String
    rua, String cidade) {
    this.nome = nome;
    this.cpf=cpf;
    this.email=email;
    this.endereco = new Endereco(rua, cidade);

    // Exemplo de metodo abstrato:
    // Define um contrato que toda subclasse deve obrigatoriamente
    seguir,
    // mas nao define como ele sera executado aqui na superclasse.
    public abstract String getDescricao();
}

```

1.2 Interfaces

Quando um sistema exige uma estrutura mais elaborada, a interface atua como um contrato de comportamento. Ela define um conjunto de métodos que devem obrigatoriamente ser implementados por qualquer classe que a utilize. O ponto central é que a interface foca apenas na definição do que deve ser feito, deixando a liberdade da implementação real para cada classe específica (Leite et al. (2016)).

Em Java a palavra **interface** deve ser utilizada antes da palavra **class** e do nome da interface. Diferente das classes abstratas, uma interface **não pode conter atributos de instância**, apenas constantes (variáveis estáticas e finais) e métodos abstratos. Uma classe que implementa uma interface deve utilizar a palavra-chave **implements** e fornecer implementações concretas para todos os métodos definidos na interface.

Vamos adionar uma interface chamada Tratamento em nosso exemplo da clínica veterinária (para fins didáticos). Essa interface terá um método chamado tratarAnimal() que será implementado por diferentes classes de tratamento, como TratamentoVeterinario e TratamentoEstetico.

```

public interface Tratamento {
    void tratarAnimal(Animal animal);
}

public class TratamentoVeterinario implements Tratamento {
    @Override
    public void tratarAnimal(Animal animal) {
        // Implementacao do tratamento veterinario
        System.out.println("Tratando o animal " + animal.getNome()
            + " com cuidados veterinarios.");
    }
}

public class TratamentoEstetico implements Tratamento {

```

```
@Override
public void tratarAnimal(Animal animal) {
    // Implementacao do tratamento estetico
    System.out.println("Tratando o animal " + animal.getNome()
        + " com cuidados esteticos.");
}
}
```

1.3 Métodos estáticos e default

Os métodos estáticos e default foram introduzidos em interfaces a partir do Java 8, permitindo que as interfaces tenham implementações de métodos. A implementação de métodos estáticos pode ser observado na seção ?? do capítulo anterior.

Logo, com a chegada dos métodos default e static, as interfaces ganharam recursos que aproximam muito a sua utilização prática da de uma classe abstrata, permitindo o compartilhamento de código pronto. No entanto, elas continuam tendo diferenças arquiteturais fundamentais que definem quando usar cada uma.

Observação

Nas interfaces, os métodos são implicitamente públicos e abstratos, enquanto nas classes abstratas, os métodos podem ter diferentes níveis de acesso e podem ser concretos ou abstratos.

1.3.1 Herança múltipla

A herança múltipla não é suportada em Java para classes, mas é possível implementar múltiplas interfaces em uma única classe. Isso permite que uma classe herde o comportamento de várias interfaces.

Um exemplo prático seria uma classe que implementa tanto a interface `Tratamento` quanto a interface `Autenticavel`, permitindo que ela trate animais e também gerencie autenticação de usuários.

1.4 Exemplo

Para integrar os conceitos de orientação a objetos na estrutura do diagrama, podemos transformar a superclasse `Pessoa` em uma classe abstrata (forçando um método de exibição) e introduzir uma interface `Autenticavel` para definir regras de acesso ou credenciais ao sistema.

```
interface Autenticavel {
    // Constante implícita
    int NIVEL_ACESSO_PADRAO = 1;

    boolean autenticar(String credencial);
}
```

1 Mecanismos de abstração

```
// Método default (Java 8+)
default void exibirCabecalhoSeguranca() {
    System.out.println("[Sistema Veterinário - Acesso Restrito
                        ]");
}
}
```

```
class Endereco {
    private String rua;
    private String cidade;

    public Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }

    @Override
    public String toString() {
        return rua + ", " + cidade;
    }
}
```

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    protected Endereco endereco; // Composição (1 para 1)

    public Pessoa(String nome, String cpf, String email, String
        rua, String cidade) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        this.endereco = new Endereco(rua, cidade); // Composição
            estrita
    }

    // Método abstrato: obriga as subclasses a implementarem a
    // exibição de dados
    public abstract void exibirInformacoes();

    public String getNome() {
        return nome;
    }
}
```

```
class Veterinario extends Pessoa implements Autenticavel {
    private String crmv;
```

1 Mecanismos de abstração

```
public Veterinario(String nome, String cpf, String email,
    String rua, String cidade, String crmv) {
    super(nome, cpf, email, rua, cidade);
    this.crmv = crmv;
}

@Override
public void exibirInformacoes() {
    System.out.println("Veterinário: " + nome + " | CRMV: " +
        crmv + " | Endereço: " + endereco);
}

@Override
public boolean autenticar(String credencial) {
    // Validação simples simulada baseada no CRMV
    return this.crmv.equals(credencial);
}

public void realizarConsulta(Consulta consulta) {
    System.out.println("O veterinário " + nome + " está
        realizando a consulta.");
}
}
```

```
class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexo; // 0 para Femea, 1 para Macho

    public Pet(String nomeAnimal, int idadeAnimal, int sexo) {
        this.nomeAnimal = nomeAnimal;
        this.idadeAnimal = idadeAnimal;
        this.sexo = sexo;
    }

    public String getNomeAnimal() {
        return nomeAnimal;
    }
}
```

```
class Consulta {
    private String data;
    private List<Pet> petsAtendidos;
    private Veterinario veterinario; // Associação

    public Consulta(String data, Veterinario veterinario) {
        this.data = data;
        this.veterinario = veterinario;
    }
}
```

1 Mecanismos de abstração

```
        this.petsAtendidos = new ArrayList<>();
    }

    public void adicionarPet(Pet pet) {
        this.petsAtendidos.add(pet);
    }

    public void finalizarConsulta() {
        System.out.println("Consulta na data: " + data);
        veterinario.exibirInformacoes();
        System.out.println("Pets atendidos nesta sessão:");
        for (Pet p : petsAtendidos) {
            System.out.println(" - " + p.getNomeAnimal());
        }
    }
}
```

```
public class Principal {
    public static void main(String[] args) {
        // Instanciando o Veterinário (Pessoa abstrata +
        // Autenticavel)
        Veterinario vet = new Veterinario("Dr. Carlos", "
            123.456.789-00", "carlos@vet.com", "Rua das Flores, 100
            ", "Ponta Grossa", "CRMV-PR 9876");

        // Testando métodos da interface e da classe abstrata
        vet.exibirCabecalhoSeguranca();
        if (vet.autenticar("CRMV-PR 9876")) {
            System.out.println("Acesso autorizado para o veteriná
                rio!");
        }

        // Criando Pets (Agregação)
        Pet pet1 = new Pet("Mel", 3, 0);
        Pet pet2 = new Pet("Thor", 5, 1);

        // Criando a Consulta e associando o veterinário
        Consulta consulta = new Consulta("03/09/2026", vet);
        consulta.adicionarPet(pet1);
        consulta.adicionarPet(pet2);

        consulta.finalizarConsulta();
    }
}
```

O diagrama abaixo ilustra a relação entre as classes e a interface, destacando a herança, implementação e associações.

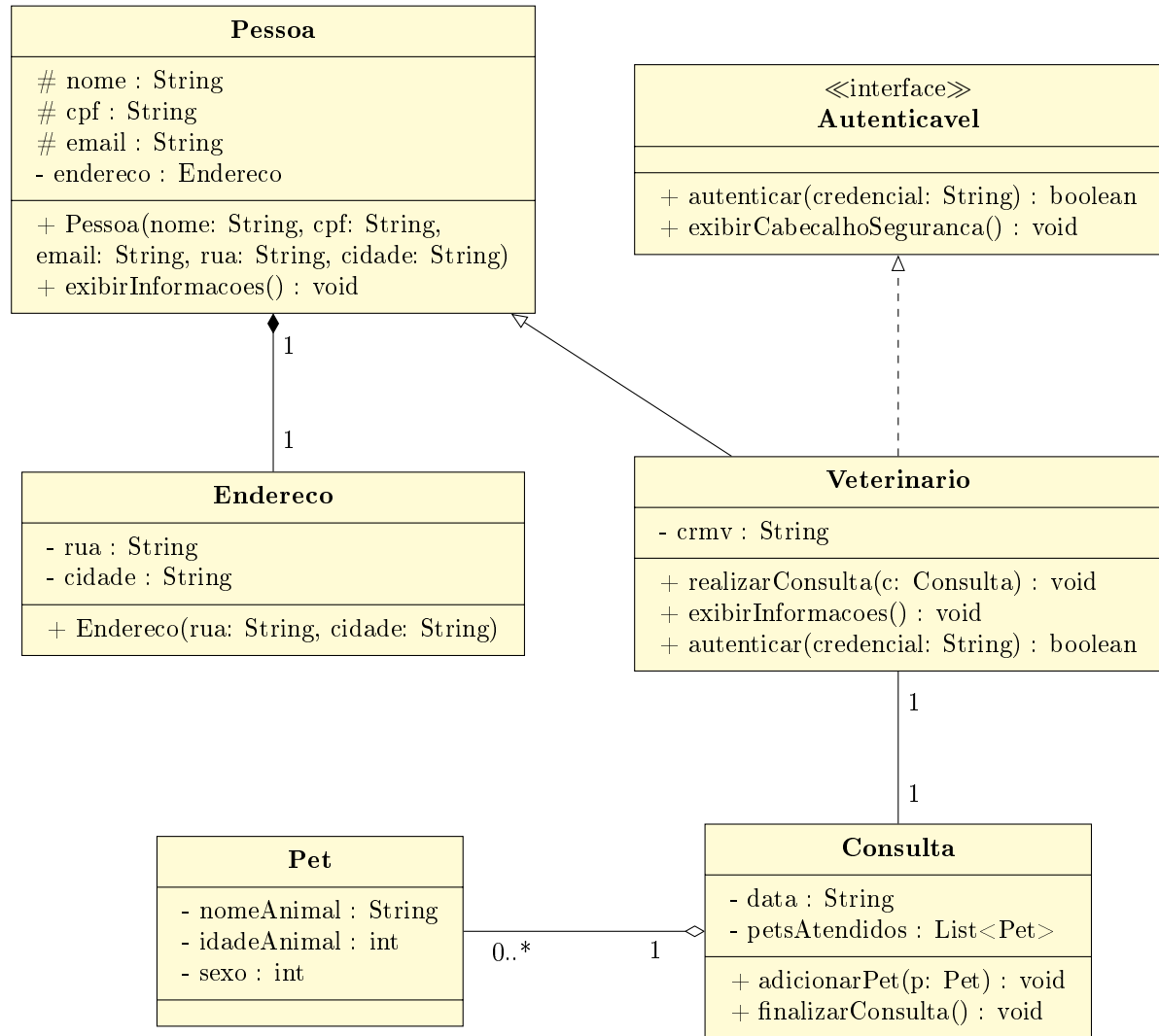


Figura 1.1: Diagrama UML atualizado incorporando a interface **Autenticavel** e a classe abstrata **Pessoa**.

1.5 Comparação entre classes abstratas e interfaces

Abaixo está uma tabela comparativa entre classes abstratas e interfaces, destacando suas principais diferenças e usos:

1 Mecanismos de abstração

Característica	Classe Abstrata	Interface
Instanciação	Não pode ser instanciada	Não pode ser instanciada
Atributos	Pode ter atributos de instância	Apenas constantes (static final)
Métodos	Pode ter métodos concretos e abstratos	Apenas métodos abstratos (Java 8+ permite métodos default e static)
Herança	Pode estender apenas uma classe abstrata	Pode implementar múltiplas interfaces
Construtores	Pode ter construtores	Não pode ter construtores

Tabela 1.1: Comparação entre Classes Abstratas e Interfaces

Referências Bibliográficas

Leite, T. et al. (2016). *Orientação a Objetos: Aprenda seus conceitos e suas aplicabilidades de forma efetiva*. Editora Casa do Código.